

Unified reference: MSG / ABB / SSG implementations across the hymeko stack

Csaba Hajdu*

2026-06-04

Scope. A single-document map of the three implementations of the Friedler 1992 MSG / ABB / SSG trio that the `hymeko` codebase carries, their file-level pointers, and the bridges between them. Companion to the 2026-05-18 dossier for Jean Pimentel (`reports/2026-05-18-pimentel-abb-ssg-msg-dossier.md`): the dossier explained *what the algorithms are*; this document explains *where they live in the source tree, how they are wired together, and what tests / CLI tools / Python bindings expose them*. Updated 2026-06-04 with the walk-side trio that landed on 2026-06-03.

1 Three application domains, one algorithmic trio

The trio is applied at three layers of the stack (Table 1).

Layer	Domain	Score / cost	ABB semantics	SSG semantics
P-graph thesis	PSE: subset of operating units producing required materials from raws	sum of per-unit fixed costs $c(u)$	cost-minimal $O' \subseteq O_{\text{MSG}}$ (or top-K)	enumerate every feasible O'
Signed-cycle	HSiKAN cycle pool	scorer over (vs , signs): balance, fraction-negative, entropy, sign-product-abs	top-K cycles by scorer, admissible-UB DFS prune	(deferred)
Signed-walk (2026-06-03)	HSiKAN walk pool	same scorer family	top-K walks by scorer, admissible-UB DFS prune	Pareto-subset over (primary, secondary) scorer pair

Table 1: The three application domains.

The score / cost type differs (cost-to-minimise on P-graphs; score-to-maximise on cycles and walks), but the algorithmic structure is identical: MSG produces the feasible / candidate population; ABB prunes via an admissible bound against a running incumbent or top-K heap; SSG returns either every feasible structure (P-graph) or the Pareto-optimal subset (walks).

2 Algorithmic invariants

2.1 MSG — candidate generation with structural feasibility filter

The MSG layer returns structures (operating-unit subsets, cycles, walks) that pass the layer’s feasibility test. For P-graphs the test is the canonical Friedler axiom set (A1–A5; `hymeko_pgraph/src/axioms.rs`). For signed cycles and walks the test is graph topology plus simple-path constraint (no vertex revisit).

*Széchenyi István Egyetem, Győr, Hungary; `gemeauxrapace@gmail.com`

When the candidate space is too large to materialise (e.g. a 4×10^9 -walk DFS on `bitcoin_alpha` `walk_len=4`), MSG returns a uniform random sample of size `cap` via reservoir sampling. The walk-side Rust path uses Vitter Algorithm L; the Python reference falls back to Algorithm R when the Rust extension is unavailable.

2.2 ABB — pruned DFS with admissible bound

Every ABB layer maintains an *incumbent* — the best feasible solution found so far (P-graph) or the worst-of-top-K (cycles, walks). At every DFS extension the layer computes an *admissible upper bound* on the score that any completion of the current prefix could achieve. If the bound is dominated by the incumbent, the entire subtree is pruned.

The admissibility postcondition is the load-bearing correctness invariant:

$$\forall \text{ closed structure completing prefix } q : \text{score}(\text{completion}) \leq \text{upper_bound}(\text{prefix state, remaining}). \quad (1)$$

A non-admissible bound silently produces a wrong top-K result. The framework surfaces admissibility as a property-based test fixture (`tests/abb_global_topk.rs::ub_admissible_*` for cycles; `signedkan_wip/tests/test_abb_msg_ssg.py::test_path_scorer_admissibility` for walks). Four lemmas in `paper/abb_msg_ssg_signed_walks/algorithms.tex` §4.4 prove admissibility for the four walk-side `PathScorer` implementations.

2.3 SSG — enumeration vs Pareto subset

P-graph SSG is *exhaustive*: it enumerates every feasible $O' \subseteq O_{\text{MSG}}$. The brute implementation (`hymeko_pgraph/src/ssg.rs`) iterates the $2^{|O_{\text{MSG}}|}$ subset lattice and refuses on $|O_{\text{MSG}}| > 30$. The canonical Friedler decision-mapping (`hymeko_pgraph/src/ssg_dm.rs`) recurses over per-material production decisions and is the only path that scales to the book’s Example 3.3 (29 units, 3,465 feasible structures).

Walk-side SSG is *Pareto subset*: it filters the ABB(primary scorer) top-K pool to the rows that are not dominated on any of a secondary set of scorers. This is the multi-objective extension. With a single objective the filter degenerates to the ABB output.

3 Implementation pointers

3.1 P-graph (Rust: `hymeko_pgraph` crate)

```
hymeko_pgraph/src/
|-- schema.rs PNodeKind enum (Material vs OperatingUnit),
| PGraphSchema with bipartite consistency
|-- lowering.rs LoweredPGraph = source -> MSG-ready IR
|-- axioms.rs A1-A5 Friedler axioms (canonical 1992
| semantics restored 2026-05-19) + certificate
|-- regime.rs Regime trait: Canonical | NoExcess |
| CostDominance | Composite (+-joined)
|-- msg.rs maximal_structure[_with_regime]
|-- ssg.rs enumerate[_with_options] -- brute  $2^{|O_{\text{MSG}}|}$ 
| subset lattice (refuses on  $|O_{\text{MSG}}| > 30$ )
|-- ssg_dm.rs enumerate -- canonical Friedler 1992
| decision-mapping (Ch. 5, Def. 5.1)
|-- abb.rs solve, solve_with_options,
| solve_with_regime, solve_top_k,
| solve_top_k_with_regime (2026-06-03)
|-- dump.rs analyze*_with_regime[_topk|_full] ->
| PgraphAnalysisJson for CLI consumers
|-- cli.rs load_pgraph, render_pgraph, render_solution,
| to_dot, to_friedler_dot (canonical PSE
| rendering, 2026-06-03), render_graphviz
```

```

'-- bin/
|  -- hymeko_pgraph_dump.rs CLI: --algorithm msg|ssg|abb
|  + --top-k N --ssg-algorithm dm
'-- pgraph.rs CLI: read | transform | solve |
                    generate (+ --style friedler --format pdf)

```

Tests: `hymeko_pgraph/tests/` — 141 tests across 12 files (including the 2026-06-03 `pimentel_distractors.rs` four-test suite).

3.2 Signed cycles (Rust: `hymeko_graph::topk_cycles`)

```

hymeko_graph/src/
|-- signed_graph.rs SignedGraph + build_csr_with_signs
|-- topk_cycles.rs BoundedScorer trait + 4 scorers +
| ~20 enumerator variants:
| - enumerate_top_k_cycles[_noprune]
| - enumerate_top_k_cycles_par_bb_batched
| - enumerate_top_k_per_vertex_cycles_*
| - tiered / adaptive / start-local /
| global
|-- unsigned_cycles/ CSR + DFS + BFS + Sink (Reservoir |
| EarlyStop | Full)
|-- friedler.rs P-graph axiom pruner for cycles
| (A0-A5) -- DFS-time structural prune
|-- pruner.rs CyclePruner trait: extend_ok + emit_ok
'-- cycle_enum.rs enumerate_simple_cycles + serial
                    reference

```

Python: `signedkan_wip/src/cycle_cache/api.py::cached_construct_k` → routes to `hymeko.enumerate` per `HSIKAN_TOPK_MODE` env var.

3.3 Signed walks (NEW, 2026-06-03)

```

hymeko_graph/src/topk_walks.rs NEW
    enumerate_top_k_walks serial DFS + admissible-UB
                                prune + min-heap top-K
    enumerate_top_k_walks_batch SoA output for zero-copy
                                PyO3
    TopKWalk, TopKWalksBatch output types

hymeko_py/src/cycles/unsigned.rs extended
    enumerate_top_k_walks_rs PyO3 binding
                                (BalanceScorer |
                                FractionNegative |
                                SignProductAbsScorer)

signedkan_wip/src/core/
|-- reservoir.py NEW ReservoirSampler (Vitter Algo R)
| + NumpyReservoirSampler
| (Algo L, numpy buffer, zero
| per-offer alloc)
|-- path_scorers.py NEW PathScorer ABC + 4 concrete:
| FractionNegative, Balance,
| SignProductAbs, ShannonEntropy
| + admissibility contract
|-- abb_walks.py NEW abb_enumerate_walks (Python
| ref + Rust delegation via
| _try_rust_top_k_walks) +
| msg_enumerate_walks +
| ssg_pareto_filter
'-- walks.py upd construct_walks_arrays uses
                    NumpyReservoirSampler when

```

```

max_walks is set (bounded
even on hymeko-missing
fallback)

signedkan_wip/src/cycle_cache/strategies.py NEW
EnumeratedArrays Output dataclass
    (v, sigma, edge_signs, is_closed)
TupleEnumerator (ABC) enumerate + cache_key_params
CycleEnumerator
WalkEnumerator = MSGWalkEnumerator Vitter-L reservoir via Rust if
    available, Python fallback
TwoCycleEnumerator (k=2) trivial wrapper of edges
TriadEnumerator (k=3)
ABBWalkEnumerator composes PathScorer; delegates to
    hymeko.enumerate_top_k_walks_rs
    when available
SSGWalkEnumerator ABB(primary) then Pareto-filter
    on (primary, secondary) scorer pair
CachedEnumerator Decorator: disk-cache + lazy pool
    wrapper
cached_construct(g, kind, ...) Single-line dispatcher

```

4 Cross-domain bridges

4.1 The BoundedScorer trait bridges cycles and walks

`hymeko_graph::topk_cycles::BoundedScorer` is defined once and used in both the cycle and the walk path:

```

pub trait BoundedScorer: Sync + Send {
    fn score(&self, vs: &[u32], signs: &[i8]) -> f64;
    fn upper_bound(&self,
        n_neg_so_far: usize,
        k_remaining: usize,
        k_len: usize) -> f64;
}

```

The contract is identical: `(vs, signs)` is the closed structure's vertex / edge-sign sequence; `(n_neg_so_far, k_remaining, k_len)` is the prefix state.

4.2 The Python PathScorer ABC mirrors BoundedScorer

```

class PathScorer(ABC):
    @abstractmethod
    def score(self, vs, signs) -> float: ...
    @abstractmethod
    def upper_bound(self, n_neg_so_far: int,
        steps_remaining: int, k_len: int) -> float: ...
    @abstractmethod
    def name(self) -> str: ...

```

The four concrete implementations (`FractionNegativeScorer`, `BalanceScorer`, `SignProductAbsScorer`, `ShannonEntropyScorer`) have the same closed-form score and upper-bound as the Rust path. `name()` is the bridge into the Rust dispatcher (`enumerate_top_k_walks_rs(..., score_kind=name)`); only the four names listed get routed to Rust, anything else falls back to the Python DFS reference.

4.3 The Friedler CyclePruner pattern will transfer to walks (planned)

The cycle-side `hymeko_graph::pruner::CyclePruner` trait (`extend_ok` during DFS + `emit_ok` at closure) is the structural analogue of Friedler's two-stage check. The walk-side currently hard-codes

the only structural check (no vertex revisit) inside the DFS; a planned refactor lifts this to a WalkPruner trait symmetric to CyclePruner. See docs/plans/2026-06-03-abb-msg-ssg-walk-enumeration/p

5 Tests and cargo conformance

Suite	File	N	Purpose
P-graph book conformance	hymeko_pgraph/tests/book_validation.rs	5	Friedler / Orosz / Pimentel-Losada book
P-graph decision-mapping SSG	hymeko_pgraph/tests/ssg_decision_mapping.rs	1	Including Ex. 3.3 = 3,465 structures
Pimentel distractor fixture	hymeko_pgraph/tests/pimentel_distractor.rs	1	MSG=7, SSG=19, ABB top-3 = 9/12
Walk ABB Rust	hymeko_graph/src/topk_walks.rs::tests	8	Brute-force equivalence over walk_len
Walk MSG / ABB / SSG Python	signedkan_wip/tests/test_abb_msg_ssg.py	10	Admissibility, ABB-vs-brute, SSG Path
Reservoir samplers	signedkan_wip/tests/test_reservoir.py	20	Algorithm R + L correctness
Strategy adapters	signedkan_wip/tests/test_tuple_enumerators.py	18	Strategy + Adapter round-trips, legacy
Cycle cache	signedkan_wip/tests/test_cycle_cache.py	13	LazyCyclePool, pack/unpack symmetry
malloc_trim helper	signedkan_wip/tests/test_malloc_trim.py	5	Glibc heap release
Pre-existing Rust	hymeko_graph/src/*.rs::tests	~75	Cycle enumerators, balance pruners, I

Table 2: Test inventory across the three layers.

Total reproducible: 141 hymeko_pgraph tests + 113 hymeko_graph tests + 72 signedkan_wip tests = **326 tests** across the three layers, all green at the 2026-06-03 freeze.

6 CLI surface (reproducible without Python imports)

6.1 P-graph analysis

```
cargo build -p hymeko_pgraph --bin hymeko_pgraph_dump

./target/debug/hymeko_pgraph_dump <file.hymeko|.pgip> \
  --algorithm msg|ssg|abb \
  [--ssg-algorithm brute|decision-mapping] # 2026-06-03
  [--top-k N] # 2026-06-03
  [--regime canonical|no-excess|cost-dominance[+...]]
  [--strict-no-excess]
  [--weights "w1,...,wD"] # multi-objective ABB
  [--write-pgip out.pgip]
```

6.2 P-graph rendering

```
cargo build -p hymeko_pgraph --bin pgraph

./target/debug/pgraph generate <file.hymeko|.pgip> \
  --format dot|png|svg|pdf|pgip \
  [--style default|friedler|pse] # 2026-06-03
  [--out PATH]
```

6.3 Book conformance suite

```
python scripts/pgraph/run_examples.py
python scripts/pgraph/run_examples.py --regimes
```

6.4 HSiKAN cycle / walk pool (Python)

```
from signedkan_wip.src.cycle_cache import cached_construct

# MSG (uniform reservoir sample)
pool = cached_construct(g, "msg_walk",
                        walk_len=4, max_walks=100_000)

# ABB (top-K by scorer)
pool = cached_construct(
    g, "abb_walk",
    walk_len=4, top_k=10_000,
    scorer_name="balance",
)

# SSG (Pareto subset on multi-axis)
pool = cached_construct(
    g, "ssg_walk",
    walk_len=4, top_k=10_000,
    primary_scorer="balance",
    secondary_scorer="entropy",
)
```

7 Empirical anchors

- **Pimentel benchmark** (reports/2026-06-03-pimentel-benchmark-reply.{md,tex,pdf}): 10-unit distractor problem reproduces MSG = 7, SSG = 19, ABB top-3 = (9, 12, 13); canonical PSE renderings in App. C.
- **Friedler / Orosz / Pimentel-Losada book** (scripts/pgraph/run_examples.py): 8/8 canonical examples (Chapter 3.2, 3.3, 4.1, 4.3, 5.1, 6.1, 14.1, HDA, methanol) match.
- **Signed-cycle ABB** (reports/2026-05-10-abb-global-topk.md): 25× wall reduction on Epinions $k = 4$ $K = 10^4$.
- **Signed-walk ABB** (this paper, §6 forthcoming): Slashdot 5-seed edge_cr Komondor audit reproduces published SOTA $0.9067 \pm .0029$ (chain 13885723: {0.9098, 0.9049, 0.9008, 0.9091, 0.9043} mean $0.9058 \pm .0033$ within $+1\sigma$); BA + OTC + Epinions in flight.

8 Open work

1. **WalkPruner trait** symmetric to CyclePruner — currently the simple-walk no-revisit check is hard-coded in `hymeko_graph::topk_walks::dfs`; lifting it to a trait would parallel the cycle-side `friedler::FriedlerAxiomPruner` and make walk-side regime composition possible.
2. **Rayon-parallel walks** — the current Rust DFS is serial; the cycle-side `enumerate_top_k_per_vertex_cycl` pattern (per-vertex DFS + thread-local heap + atomic global incumbent) is the template.
3. **Python binding for the P-graph trio** — the CLI binary is the only integration surface for non-Rust callers. A PyO3 wrapper exposing MSG / SSG / ABB / regime is on the planned-work list.
4. **SSG-walk skyline** — `pareto_filter` is brute $O(N^2 D)$; sort-and-sweep (Kung–Luccio–Preparata 1975) lowers to $O(N \log^{D-1} N)$ for large pools.

A Algorithm code listings (the actual implementation)

This appendix carries the load-bearing implementations of MSG, ABB, and SSG verbatim from the source tree. The cross-reference is by file path; each block is the canonical entry point for its

respective layer.

A.1 P-graph MSG — bipartite fixpoint reduction

Source: `hymeko_pgraph/src/msg.rs`. The function performs a two-phase fixpoint reduction over the bipartite IR: a forward feasibility pass that drops units producing a raw material or consuming an unavailable input, then a backward composition pass keeping only units that reach a required product. A regime refinement step on the output handles `NoExcess` / `CostDominance`.

```
pub fn maximal_structure_with_regime(
    p: &LoweredPGraph,
    regime: &dyn crate::regime::Regime,
) -> MaximalStructure {
    // -- Reduction phase (forward feasibility; admits cycles) --
    //
    // Iterate to a fixpoint, removing (1) units that produce a raw
    // material (axiom S2: raws have no producer), and (2) units
    // with an input that is neither raw nor produced by *any*
    // surviving unit. Availability uses "produced by some survivor"
    // -- NOT reachability-from-raws -- so a structurally valid
    // cycle (whose members mutually produce each other's inputs)
    // survives.
    let mut units: BTreeSet<DeclId> = p.units.clone();
    loop {
        let before = units.len();
        units.retain(|u| !p.outputs(*u).iter()
            .any(|m| p.raws.contains(m)));
        let mut available: BTreeSet<DeclId> = p.raws.clone();
        for u in &units {
            available.extend(p.outputs(*u).iter().copied());
        }
        units.retain(|u| p.inputs(*u).iter()
            .all(|m| available.contains(m)));
        if units.len() == before { break; }
    }

    // -- Composition phase (backward reachability from products) --
    //
    // Least fixpoint: a unit is kept iff it produces a required
    // material; including it makes its non-raw inputs required in
    // turn. Forward-feasible units that reach no product (dead-end
    // producers) are dropped here.
    let mut kept: BTreeSet<DeclId> = BTreeSet::new();
    let mut needed: BTreeSet<DeclId> = p.products.clone();
    loop {
        let new: Vec<DeclId> = units
            .iter().copied()
            .filter(|u| !kept.contains(u)
                && p.outputs(*u).iter().any(|m| needed.contains(m)))
            .collect();
        if new.is_empty() { break; }
        for u in new {
            kept.insert(u);
            needed.extend(p.inputs(u).iter().copied()
                .filter(|m| !p.raws.contains(m)));
        }
    }

    // -- Regime refinement (Canonical = identity; NoExcess =
    // no-waste filter) --
    let units = regime.refine_maximal(p, kept);
    // ... (materials bookkeeping omitted)
    MaximalStructure { units, materials }
}
```

```

/// Forward closure: smallest set C \supseteq R such that whenever
/// a unit in 'units' has all its inputs in C, its outputs are
/// added. Used by both the ABB reachability bound and SSG's
/// per-material decision recursion.
pub fn close_producible(
  p: &LoweredPGraph,
  units: &BTreeSet<DeclId>,
  rows: &BTreeSet<DeclId>,
) -> BTreeSet<DeclId> {
  let mut c: BTreeSet<DeclId> = rows.clone();
  loop {
    let before = c.len();
    for u in units {
      if p.inputs(*u).iter().all(|m| c.contains(m)) {
        c.extend(p.outputs(*u).iter().copied());
      }
    }
    if c.len() == before { return c; }
  }
}

```

A.2 P-graph SSG — canonical decision-mapping recursion

Source: `hymeko_pgraph/src/ssg_dm.rs`. The Friedler 1992 algorithm (Ch. 5, Def. 5.1, Fig. 5.13) generates each solution-structure exactly once by recursing over per-material production decisions. The producer query $\Delta(x) = \{u : x \in \text{outputs}(u)\}$ is queried over the `LoweredPGraph::producers` accessor.

```

pub fn enumerate(
  p: &LoweredPGraph,
  msg: &MaximalStructure,
) -> Vec<SolutionStructure> {
  enumerate_with_options(p, msg, SsgDmOptions::default()).structures
}

pub fn enumerate_with_options(
  p: &LoweredPGraph,
  msg: &MaximalStructure,
  opts: SsgDmOptions,
) -> SsgDmResult {
  let mut e = Enumerator {
    p, units: &msg.units, out: Vec::new(),
    max_structures: opts.max_structures, capped: false,
  };
  // Initial work-list: required products that are not themselves
  // raw (a raw product needs no production decision).
  let p0: BTreeSet<DeclId> = p.products.iter()
    .filter(|m| !p.rows.contains(m))
    .copied().collect();
  e.recurse(&BTreeSet::new(), &BTreeSet::new(),
    &BTreeSet::new(), &p0);
  SsgDmResult { structures: e.out, capped: e.capped }
}

impl Enumerator<'_> {
  /// \Delta(x) restricted to the maximal structure.
  fn delta(&self, x: DeclId) -> BTreeSet<DeclId> {
    self.p.producers(x).iter()
      .filter(|u| self.units.contains(u))
      .copied().collect()
  }

  fn recurse(

```



```

    &mut self,
    included: &BTreeSet<DeclId>,
    excluded: &BTreeSet<DeclId>,
    decided: &BTreeSet<DeclId>,
    work: &BTreeSet<DeclId>,
) {
    if self.capped { return; }

    // Pick next undecided material; work-list empty
    // => 'included' is a complete solution-structure.
    let Some(&x) = work.iter().next() else {
        if self.max_structures != 0
            && self.out.len() >= self.max_structures {
            self.capped = true; return;
        }
        self.out.push(SolutionStructure {
            units: included.clone(),
        });
        return;
    };

    let mut rest = work.clone(); rest.remove(&x);
    let delta: Vec<DeclId> = self.delta(x).into_iter().collect();
    // No producer in MSG => infeasible branch; prune.
    if delta.is_empty() { return; }
    let k = delta.len();
    debug_assert!(k <= 31);

    // Candidate decisions: every non-empty subset of \Delta(x).
    for mask in 1u32..(1u32 << k) {
        let chosen: BTreeSet<DeclId> = (0..k)
            .filter(|i| (mask >> i) & 1 == 1)
            .map(|i| delta[i]).collect();
        let dropped: BTreeSet<DeclId> = delta.iter()
            .filter(|u| !chosen.contains(u)).copied().collect();

        // Consistency tests (book Fig. 5.13):
        // include-consistency: chosen \cap excluded = \emptyset
        // exclude-consistency: dropped \cap included = \emptyset
        if chosen.iter().any(|u| excluded.contains(u)) {
            continue;
        }
        if dropped.iter().any(|u| included.contains(u)) {
            continue;
        }

        let mut included2 = included.clone();
        included2.extend(&chosen);
        let mut excluded2 = excluded.clone();
        excluded2.extend(&dropped);
        let mut decided2 = decided.clone();
        decided2.insert(x);

        // Including a unit makes each of its non-raw inputs
        // a material that must itself be produced (axiom S4 /
        // forward feasibility). Queue inputs not yet decided.
        let mut work2 = rest.clone();
        for u in &chosen {
            for m in self.p.inputs(*u) {
                if !self.p.raws.contains(m)
                    && !decided2.contains(m) {
                    work2.insert(*m);
                }
            }
        }
    }
}

```

```

    }

    self.recurse(&included2, &excluded2, &decided2, &work2);
    if self.capped { return; }
  }
}
}

```

A.3 P-graph ABB — DFS with inclusion + reachability bounds

Source: `hymeko_pgraph/src/abb.rs`. The recursion is a two-bound branch-and-bound: **inclusion bound** (partial cost $\geq$ incumbent) and **reachability bound** (optimistic remaining set fails to produce a required product). At every depth the algorithm branches into include / exclude on the next unit.

```

pub fn solve_with_regime(
  p: &LoweredPGraph,
  msg: &MaximalStructure,
  opts: AbbOptions,
  regime: &dyn crate::regime::Regime,
) -> Option<AbbSolution> {
  let order: Vec<DeclId> = msg.units.iter().copied().collect();
  let mut state = SearchState {
    order,
    included: BTreeSet::new(),
    excluded: BTreeSet::new(),
    cost: 0.0,
    best: None,
    explored: 0,
    pruned_by_inclusion: 0,
    pruned_by_reachability: 0,
    opts,
    regime,
  };
  branch(p, msg, &mut state, 0);
  state.best.map(|(units, cost)| AbbSolution {
    units, cost,
    explored: state.explored,
    pruned_by_inclusion: state.pruned_by_inclusion,
    pruned_by_reachability: state.pruned_by_reachability,
  })
}

fn branch(
  p: &LoweredPGraph,
  msg: &MaximalStructure,
  s: &mut SearchState<'_,>,
  depth: usize,
) {
  if s.opts.max_explored != 0
    && s.explored >= s.opts.max_explored { return; }
  s.explored += 1;

  // -- Bound 1: inclusion bound --
  if let Some((_, best_cost)) = &s.best {
    if s.cost >= *best_cost {
      s.pruned_by_inclusion += 1; return;
    }
  }

  // -- Bound 2: reachability bound --
  //
  // The optimistic remaining-units set is included \cup undecided.

```

```

// If even with everything still on the table we can't produce
// every required product, this branch is infeasible.
let mut optimistic: BTreeSet<DeclId> = s.included.clone();
for u in &s.order[depth..] {
    if !s.excluded.contains(u) {
        optimistic.insert(*u);
    }
}
let producible = close_producible(p, &optimistic, &p.rows);
if !p.products.iter().all(|m| producible.contains(m)) {
    s.pruned_by_reachability += 1; return;
}

// -- Leaf: decide --
if depth == s.order.len() {
    if is_feasible_with_regime(p, &s.included, s.regime) {
        let candidate = (s.included.clone(), s.cost);
        match &s.best {
            None => s.best = Some(candidate),
            Some(_, bc) => if s.cost < *bc =>
                s.best = Some(candidate),
            _ => {}
        }
    }
    return;
}

// -- Branch: include first (greedy: an incumbent shows up
// early, tightening the inclusion bound for the exclude
// branch).
let u = s.order[depth];
let cu = effective_cost(p, &s.opts, u);
s.included.insert(u);
s.cost += cu;
branch(p, msg, s, depth + 1);
s.included.remove(&u);
s.cost -= cu;

// -- Branch: exclude --
s.excluded.insert(u);
branch(p, msg, s, depth + 1);
s.excluded.remove(&u);
}

/// Top-k variant: enumerate via the decision-mapping SSG and pick
/// the K cheapest admissible structures.
pub fn solve_top_k_with_regime(
    p: &LoweredPGraph,
    msg: &MaximalStructure,
    k: usize,
    opts: AbbOptions,
    regime: &dyn crate::regime::Regime,
) -> Vec<AbbSolution> {
    if k == 0 { return Vec::new(); }
    let ssg = crate::ssg_dm::enumerate(p, msg);
    let mut scored: Vec<(f64, BTreeSet<DeclId>)> = ssg
        .into_iter()
        .filter(|s| regime.structure_admissible(p, &s.units))
        .map(|s| {
            let cost: f64 = s.units.iter()
                .map(|u| effective_cost(p, &opts, *u)).sum();
            (cost, s.units)
        })
        .collect();

```

```

scored.sort_by(|a, b| a.0.partial_cmp(&b.0)
                .expect("non-finite cost"));
scored.into_iter().take(k)
    .map(|(cost, units)| AbbSolution {
        units, cost,
        explored: 0, pruned_by_inclusion: 0,
        pruned_by_reachability: 0,
    })
    .collect()
}

```

A.4 Signed-walk ABB — DFS with min-heap top-K and admissible-UB prune

Source: `hymeko_graph/src/topk_walks.rs`. Mirrors the cycle ABB but for open walks: no closure check, canonical-form filter `walk[0] <= walk[walk_len]`, min-heap of size K on `BoundedScorer.score`.

```

pub fn enumerate_top_k_walks<S: BoundedScorer>(
    graph: &SignedGraph,
    walk_len: usize,
    top_k: usize,
    scorer: &S,
) -> Vec<TopKWalk> {
    if walk_len == 0 || top_k == 0 { return Vec::new(); }
    if walk_len > MAX_INLINE_WALK_LEN { return Vec::new(); }
    let (row_ptr, col_idx, signs_csr) = graph.build_csr_with_signs();
    let n = graph.n_nodes as usize;
    let mut visited = vec![false; n];
    let mut path: Vec<u32> = Vec::with_capacity(walk_len + 1);
    let mut signs: Vec<i8> = Vec::with_capacity(walk_len);
    let mut heap: BinaryHeap<HeapEntry> =
        BinaryHeap::with_capacity(top_k + 1);

    for start in 0..(n as u32) {
        path.clear(); signs.clear();
        path.push(start);
        visited[start as usize] = true;
        dfs(start, &row_ptr, &col_idx, &signs_csr,
            walk_len, top_k, scorer,
            &mut path, &mut signs, &mut visited, &mut heap);
        visited[start as usize] = false;
    }

    let mut out: Vec<TopKWalk> = heap.into_iter()
        .map(|e| (e.score,
            e.walk_slice().to_vec(),
            e.signs_slice().to_vec()))
        .collect();
    out.sort_by(|a, b| b.0.partial_cmp(&a.0)
                .unwrap_or(Ordering::Equal));
    out
}

fn dfs<S: BoundedScorer>(
    start: u32,
    row_ptr: &[u32], col_idx: &[u32], signs_csr: &[i8],
    walk_len: usize, top_k: usize, scorer: &S,
    path: &mut Vec<u32>, signs: &mut Vec<i8>,
    visited: &mut [bool], heap: &mut BinaryHeap<HeapEntry>,
) {
    if signs.len() == walk_len {
        // Complete walk: canonical-form filter then offer.
        let last = *path.last().unwrap();
    }
}

```

```

    if start <= last {
        let sc = scorer.score(path, signs);
        if heap.len() < top_k {
            heap.push(HeapEntry::from_slices(sc, path, signs));
        } else if let Some(worst) = heap.peek() {
            let candidate =
                HeapEntry::from_slices(sc, path, signs);
            if candidate.cmp_preference(worst)
                == Ordering::Greater {
                heap.pop();
                heap.push(candidate);
            }
        }
    }
    return;
}

// -- ABB upper-bound prune: skip the entire subtree if
// the optimistic best completion of the current prefix
// cannot displace the worst entry in the heap.
if heap.len() >= top_k {
    let n_neg_so_far = signs.iter()
        .filter(|&s| s < 0).count();
    let k_remaining = walk_len - signs.len();
    let ub = scorer.upper_bound(
        n_neg_so_far, k_remaining, walk_len);
    if let Some(worst) = heap.peek() {
        if ub <= worst.score { return; }
    }
}

let tail = *path.last().unwrap();
let s_idx = row_ptr[tail as usize] as usize;
let e_idx = row_ptr[tail as usize + 1] as usize;
for ei in s_idx..e_idx {
    let nxt = col_idx[ei];
    if visited[nxt as usize] { continue; }
    let edge_sign = signs_csr[ei];
    path.push(nxt);
    signs.push(edge_sign);
    visited[nxt as usize] = true;
    dfs(start, row_ptr, col_idx, signs_csr,
        walk_len, top_k, scorer,
        path, signs, visited, heap);
    path.pop(); signs.pop();
    visited[nxt as usize] = false;
}
}

```

A.5 NumpyReservoirSampler — Vitter Algorithm L on a numpy buffer

Source: `signedkan_wip/src/core/reservoir.py`. The walk-side MSG primitive: zero per-offer Python allocation, expected RNG cost $O(K \log(N/K))$ via Algorithm L. The hot path on a rejected offer is a single integer compare and an increment.

```

class NumpyReservoirSampler:
    def __init__(
        self, cap: int, k: int,
        dtype: np.dtype | type = np.int32, seed: int = 0,
    ):
        if cap < 0: raise ValueError(f"cap must be >= 0, got {cap}")
        if k < 0: raise ValueError(f"k must be >= 0, got {k}")
        self.cap = int(cap); self.k = int(k)

```

```

self.dtype = np.dtype(dtype)
self.buf = np.zeros(
    (max(self.cap, 0), self.k), dtype=self.dtype)
self.seen = 0
self.rng = random.Random(int(seed))
# Algorithm L state. 'W' shrinks across selections;
# 'next_select' is the next stream index to land on.
if self.cap > 0:
    self.W = math.exp(math.log(self.rng.random()) / self.cap)
    self.next_select = (
        self.cap - 1 + 1
        + int(math.log(self.rng.random())
              / math.log(1 - self.W))
    )
else:
    self.W = 0.0; self.next_select = 0

def offer(self, seq) -> None:
    """Submit one length-k sequence (list / tuple / 1-D numpy).

    Hot path on rejection: one integer compare + one increment.
    No allocation, no RNG call. Only when 'seen == next_select'
    (or in the pre-fill phase) does the sampler do real work.
    """
    if self.cap == 0:
        self.seen += 1; return
    if self.seen < self.cap:
        # Pre-fill phase: always accept, direct row assignment.
        # numpy handles the list/tuple -> row copy in C.
        self.buf[self.seen] = seq
    elif self.seen == self.next_select:
        # Algorithm L selection: replace a uniform-random
        # reservoir row, then advance 'W' and compute the
        # next stream index. Two RNG draws regardless of
        # stream length.
        j = self.rng.randint(0, self.cap - 1)
        self.buf[j] = seq
        self.W *= math.exp(
            math.log(self.rng.random()) / self.cap)
        try:
            step = int(math.log(self.rng.random())
                      / math.log(1 - self.W))
        except (ValueError, ZeroDivisionError):
            # log(1 - W) = -inf when W -> 1; never select again.
            step = 1 << 62
        self.next_select += 1 + step
    # else: between selections -- just bump the counter.
    self.seen += 1

def to_array(self) -> np.ndarray:
    """Return the (min(cap, seen), k) view of the reservoir.
    Slice of the preallocated buffer -- no copy."""
    n_kept = min(self.cap, self.seen)
    return self.buf[:n_kept]

```

A.6 PathScorer ABC + the four concrete scorers

Source: `signedkan_wip/src/core/path_scorers.py`. Each scorer must satisfy the admissibility postcondition; the four admissibility proofs appear in §5 of the paper.

```

class PathScorer(ABC):
    """Score + admissible upper-bound contract for ABB pruning."""

```

```

@abstractmethod
def score(self, vs, signs) -> float: ...

@abstractmethod
def upper_bound(
    self, n_neg_so_far: int,
    steps_remaining: int, k_len: int,
) -> float: ...

@abstractmethod
def name(self) -> str: ...

class FractionNegativeScorer(PathScorer):
    def score(self, vs, signs) -> float:
        if not len(signs): return 0.0
        n_neg = sum(1 for s in signs if s < 0)
        return n_neg / len(signs)

    def upper_bound(
        self, n_neg_so_far, steps_remaining, k_len,
    ) -> float:
        if k_len <= 0: return 0.0
        return (n_neg_so_far + steps_remaining) / k_len

    def name(self) -> str: return "fraction_negative"

class BalanceScorer(PathScorer):
    def score(self, vs, signs) -> float:
        if not len(signs): return 1.0
        out = 1
        for s in signs: out *= s
        return float(out)

    def upper_bound(
        self, n_neg_so_far, steps_remaining, k_len,
    ) -> float:
        return 1.0 # trivially admissible; product in {-1, +1}

    def name(self) -> str: return "balance"

class SignProductAbsScorer(PathScorer):
    def score(self, vs, signs) -> float:
        return 1.0 if len(signs) > 0 else 0.0

    def upper_bound(
        self, n_neg_so_far, steps_remaining, k_len,
    ) -> float:
        return 1.0

    def name(self) -> str: return "sign_product_abs"

class ShannonEntropyScorer(PathScorer):
    """Per-vertex frequency Shannon entropy of the walk."""

    def score(self, vs, signs) -> float:
        if len(vs) == 0: return 0.0
        counts = Counter(int(v) for v in vs)
        total = sum(counts.values())
        if total == 0: return 0.0
        return -sum(

```

```

        (c / total) * math.log(c / total)
    for c in counts.values()
)

def upper_bound(
    self, n_neg_so_far, steps_remaining, k_len,
) -> float:
    # k_len walk-edges -> k_len + 1 vertices when distinct.
    # Max entropy over k_len + 1 outcomes is log(k_len + 1).
    if k_len <= 0: return 0.0
    return math.log(k_len + 1)

def name(self) -> str: return "entropy"

```

A.7 Python ABBWalkEnumerator strategy + Rust delegation

Sources: `signedkan_wip/src/cycle_cache/strategies.py` (strategy class) + `signedkan_wip/src/core/abb_`
(`abb_enumerate_walks` with the Rust delegation shim).

```

class ABBWalkEnumerator(TupleEnumerator):
    """Top-K walks under a 'PathScorer' with admissible-UB DFS prune.

    Composes the scorer adapter so the same DFS engine serves
    every score family (balance, fraction_negative, entropy, ...)
    without code duplication.
    """
    def __init__(
        self, walk_len: int, top_k: int,
        scorer_name: str = "balance",
    ):
        if walk_len < 1:
            raise ValueError(f"walk_len must be >= 1, got {walk_len}")
        if top_k < 1:
            raise ValueError(f"top_k must be >= 1, got {top_k}")
        self.walk_len = int(walk_len)
        self.top_k = int(top_k)
        self.scorer_name = str(scorer_name)

    def enumerate(self, g, *, seed: int) -> EnumeratedArrays:
        from ..core.abb_walks import abb_enumerate_walks
        from ..core.path_scorers import pick_scorer
        scorer = pick_scorer(self.scorer_name)
        eu = np.ascontiguousarray(g.edges[:, 0], dtype=np.int64)
        ev = np.ascontiguousarray(g.edges[:, 1], dtype=np.int64)
        es = np.ascontiguousarray(g.signs, dtype=np.int8)
        v, edge_signs, _stats = abb_enumerate_walks(
            eu, ev, es, int(g.n_nodes),
            walk_len=self.walk_len, top_k=self.top_k,
            scorer=scorer, seed=seed,
        )
        sigma = _walks_sigma_from_edge_signs(edge_signs, self.walk_len)
        return EnumeratedArrays(
            v=v.astype(np.int32), sigma=sigma,
            edge_signs=edge_signs, is_closed=False,
        )

def abb_enumerate_walks(
    edges_u, edges_v, edge_signs, n_nodes,
    walk_len, top_k, scorer, *, seed=0,
):
    """Top-K walks under 'scorer.score' with admissible-UB DFS
    pruning. Delegates to the Rust enumerator when the hymeko

```



```

wheel is installed and the scorer has a Rust analog; falls
back to a Python DFS reference otherwise."""
if walk_len < 1: raise ValueError(...)
if top_k <= 0: return (
    np.zeros((0, walk_len+1), np.int32),
    np.zeros((0, walk_len), np.int8),
    ABBStats(),
)

# Production runner: delegate to Rust when available.
rust = _try_rust_top_k_walks(
    edges_u, edges_v, edge_signs, n_nodes,
    walk_len, top_k, scorer,
)
if rust is not None:
    walks_v, walks_signs = rust
    return walks_v, walks_signs, ABBStats(
        n_emitted=int(walks_v.shape[0]))

# Python reference path -- correctness specification used in
# tests and in the Singularity case where hymeko is missing.
# (DFS body omitted for brevity; mirrors the Rust path above.)
...

def _try_rust_top_k_walks(
    edges_u, edges_v, edge_signs, n_nodes,
    walk_len, top_k, scorer,
):
    """Attempt the Rust path; return None when unavailable so the
    Python reference takes over."""
    try:
        import hymeko as _hk
    except ImportError:
        return None
    if not hasattr(_hk, "enumerate_top_k_walks_rs"):
        return None
    name = scorer.name()
    if name not in {"balance", "fraction_negative",
                    "sign_product_abs"}:
        return None
    eu = np.ascontiguousarray(edges_u, dtype=np.uint32)
    ev = np.ascontiguousarray(edges_v, dtype=np.uint32)
    es = np.ascontiguousarray(edge_signs, dtype=np.int8)
    walks, signs, _scores = _hk.enumerate_top_k_walks_rs(
        eu.tolist(), ev.tolist(), es.tolist(),
        int(n_nodes), int(walk_len), int(top_k), name,
    )
    return (
        np.asarray(walks, dtype=np.int32),
        np.asarray(signs, dtype=np.int8),
    )

```

A.8 SSG-walk Pareto filter (multi-axis)

Source: `signedkan_wip/src/core/abb_walks.py`. The brute $O(N^2D)$ skyline filter on the ABB(primary) pool.

```

def ssg_pareto_filter(
    walks_v: np.ndarray,
    walks_signs: np.ndarray,
    score_axes: list[np.ndarray],
):

```

```

"""Subset Structure Generation -- Pareto filter walks across
multiple score axes. Higher is better on every axis (negate
the cost axis before passing if minimising).
"""

if not score_axes:
    return walks_v, walks_signs, \
        np.ones(len(walks_v), dtype=bool)
N = walks_v.shape[0]
if N == 0:
    return walks_v, walks_signs, np.zeros(0, dtype=bool)
if any(ax.shape != (N,) for ax in score_axes):
    raise ValueError(...)
scores = np.column_stack(score_axes) # (N, D)
mask = np.ones(N, dtype=bool)
# O(N^2 D) brute skyline; fine for top_k <= 10^4.
for i in range(N):
    if not mask[i]: continue
    ge = (scores >= scores[i]).all(axis=1)
    gt_any = (scores > scores[i]).any(axis=1)
    dominators = ge & gt_any
    dominators[i] = False
    if dominators.any():
        mask[i] = False
return walks_v[mask], walks_signs[mask], mask

```

B Document genealogy

This document subsumes the implementation-pointer content of the 2026-05-18 Pimentel dossier, the early P-graph brief (`reports/pgraph_hymeko_brief.tex`), the walk-side framework plans (`docs/plans/2026-06-03-*/plan.md`), the Pimentel benchmark validation trail (`reports/2026-06-03-pimentel-benchmark-validation-trail.md`), the external Pimentel reply (`reports/2026-06-03-pimentel-benchmark-reply.{md,tex,pdf}`), and the paper drafts (`paper/abb_msg_ssg_signed_walks/*.tex`).

For *what the algorithms* refer to the Pimentel dossier and the paper. For *where they live in the source tree* and *how to invoke them* this document is canonical.